NorthMart Analytical Data Platform

PA1: From Raw Files to a Kimball Star Schema in DuckDB & Parquet

Data Engineering (2026/2027) | Instituto Politécnico de Bragança (IPB / UPB)
Oral Defense & System Implementation Review

Business Scenario & Analytical Bottlenecks

Why NorthMart needed a dedicated dimensional layer

The Operational Problem (As-Is)

- **Disorganized Data Islands:**
  Analysts queried 9 separate CSV exports (~121 MB) and a nested JSON file directly.

- **Slow, Wasteful Scans:**
  Queries parsed raw CSV text on every run, repeating disk reads and type inference.

- **Inconsistent Joins:**
  Different analysts joined tables differently, producing conflicting KPI reports.

- **Dirty Operational Feeds:**
  JSON exports contained temporal duplicate seller profiles, missing SLA fields, and malformed casings.

The Engineering Solution (To-Be)

✔️ **Kimball Dimensional Star Schema:**
Single conformed fact table at order-item grain surrounded by 4 clean dimensions.

✔️ **Embedded DuckDB Engine:**
Vectorized in-memory SQL execution eliminating heavyweight cluster overhead.

✔️ **Curated Parquet Storage:**
Columnar compression reducing disk footprint by 97.1% with partition pruning.

✔️ **Automated Quality Gates:**
Surrogate keys and referential integrity assertions ensuring zero orphaned records.

Task 1: Raw Data Profiling & Empirical Findings

Conducted in DuckDB over 121.4 MB of raw CSVs and JSON

Finding 1: JSON Duplication (25 Duplicates)

The JSON export has 3,095 rows but only 3,070 unique sellers. Multiple records represent state updates over time (e.g. seller dc120d93... updated in 2017 and 2018).

Finding 2: Dirty JSON Tiers & Null SLA

profile.tier contains 9 distinct raw casing variants ('Silver ', 'SILVER', 'silver'). 30 sellers (0.97%) have completely null shipping objects.

Finding 3: CSV vs JSON Seller Discrepancies

40 sellers in the CSV master are absent from the JSON export; 15 sellers in JSON are absent from CSV orders. Required LEFT JOIN with conformed defaults.

Finding 4: Order Granularity & Lifecycle Nulls

99,441 orders generate 112,650 order items (1:N, max 21 items/order). 2,965 orders (2.98%) lack customer delivery dates (in-transit/cancelled).

Finding 5: Catalog Nulls & Missing Translations

610 products have null category names; 2 Portuguese categories lack English translations. Resolved via Kimball 'unknown' imputation.

Task 2: Kimball Star Schema Architecture

Single fact table, conformed dimensions, and explicit grain declaration

Core Architectural Specifications

1. Declared Grain (Kimball Rule):

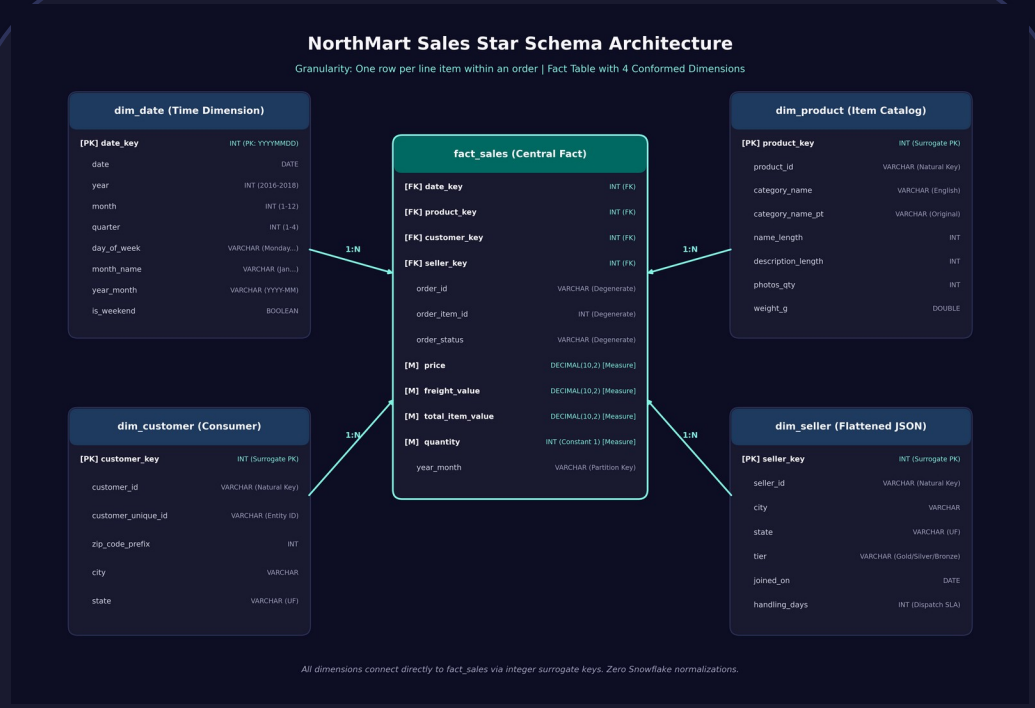
"One row per line item within an order placed on the NorthMart marketplace."

2. Fully Additive Measures:

- price, freight_value, total_item_value, quantity (constant 1)

3. Non-Obvious Modelling Decisions:

- Primary Date: Anchored to order purchase date (not delivery date, which has 2,965 nulls).
- Degenerate Dimension: order_id kept in fact table to eliminate unnecessary join tables.



Task 3: Pipeline Implementation & Quality Gates

Pure SQL transformation pipeline in DuckDB with zero-orphan referential checks

Dimensional ETL Highlights

- **Surrogate Keys:**
  Deterministic integer keys generated with ROW_NUMBER() OVER (ORDER BY natural_key). Insulates analytical schema from upstream source system key changes.
- **JSON Flattening & Deduplication:**
  Extracted nested structs via dot-notation. Deduplicated sellers with QUALIFY ROW_NUMBER() OVER (PARTITION BY seller_id ORDER BY updated_at DESC) = 1.
- **Catalog Enrichment:**
  Joined Portuguese catalog with English translations; mapped null categories to 'unknown'.
- **Preserved Fact Granularity:**
  Exactly 112,650 order items loaded into fact_sales (100% data preservation).

Referential Integrity Validation

✔️ **fact_sales -> dim_date: 0 Orphaned Keys [PASSED]**

✔️ **fact_sales -> dim_product: 0 Orphaned Keys [PASSED]**

✔️ **fact_sales -> dim_customer: 0 Orphaned Keys [PASSED]**

✔️ **fact_sales -> dim_seller: 0 Orphaned Keys [PASSED]**

✔️ **Price / Freight Measures: No Negative Values [PASSED]**

✔️ **Grain Uniqueness Check: 0 Key Collisions [PASSED]**

Task 4: Storage Footprints & Performance Benchmarks

Empirical measurement across CSV vs Parquet Unpartitioned vs Parquet Partitioned

1. Storage Footprint Comparison

- Full Raw Source (9 CSVs + 1 JSON): 121.40 MB (Baseline)
- Curated Parquet Unpartitioned (fact_sales.parquet): 3.49 MB --> 97.1% Storage Reduction
- Curated Parquet Partitioned (24 Month Directories): 3.63 MB --> 97.0% Storage Reduction

2. Query Execution Latency (Average of 5 Runs in DuckDB)

- Query 1 (Monthly Revenue Over Whole Period):
  CSV: 190.65 ms | Parquet: 11.02 ms | Partitioned: 8.14 ms --> 23.4x Faster
- Query 2 (Single Month Revenue & Items - 2018-05):
  CSV: 171.30 ms | Parquet: 6.94 ms | Partitioned: 3.07 ms --> 55.8x Faster (Partition Pruning)
- Query 3 (Top 5 Product Categories by Revenue):
  CSV: 135.20 ms | Parquet: 15.46 ms | Partitioned: 12.08 ms --> 11.2x Faster

Task 5: Architectural Justification (Warehouse vs Lake vs Lakehouse)

NorthMart is best served by a Lakehouse architecture

Data Warehouse

❌ Excessive Cost
Proprietary storage and compute licensing (Snowflake, BigQuery) is wasteful for 112k rows.

❌ Rigid Schemas
Slow to adapt to new unstructured feeds.

Data Lake

❌ Becomes a 'Data Swamp'
No schema enforcement, no ACID transactions, and no table catalogue.

❌ Unreliable Reports
Concurrent writes cause corrupt reads and divergent joins.

Data Lakehouse

✔️ Fact 1: 97.1% Storage Saving
Parquet reduces 121.4 MB raw CSV to 3.49 MB open storage.

✔️ Fact 2: 55.8x Latency Drop
Single-month queries drop from 171 ms to 3.07 ms via partition pruning.

✔️ ACID & Interoperability
Open files with zero SaaS vendor lock-in.

Oral Defense Conceptual Cheat Sheet

Essential concepts required for individual examination

Surrogate Keys

Integer keys created by the warehouse. Faster joins, isolates schema from operational changes, enables SCD Type 2.

Grain Declaration

The single sentence defining what 1 row is. Must be declared first to prevent incorrect aggregations.

Degenerate Dimension

Dimension attribute kept in the fact table without its own table (order_id). Avoids unnecessary joins.

Partition Pruning

Query engine skips irrelevant directories using file metadata. Q2 skips 23/24 month partitions in 3 ms.

Columnar Storage

Parquet reads only queried columns and skips unneeded ones. Drastically reduces I/O compared to CSV.

Idempotency

Pipeline can run repeatedly producing the identical clean state (CREATE OR REPLACE, OVERWRITE TRUE).

Summary & Reproduction Commands

Everything runs deterministically from the repository

Commands to Reproduce the Full Evaluation

Step 1 - Profile Raw Files:

```
$ duckdb < sql/profiling.sql
```

Step 2 - Build Star Schema & Curated Parquet:

```
$ duckdb northmart.duckdb < sql/build_model.sql
```

Step 3 - Run Benchmark Suite:

```
$ python3 src/run_benchmark.py
```

Step 4 - Run Automated Quality Tests:

```
$ python3 tests/run_tests.py
```